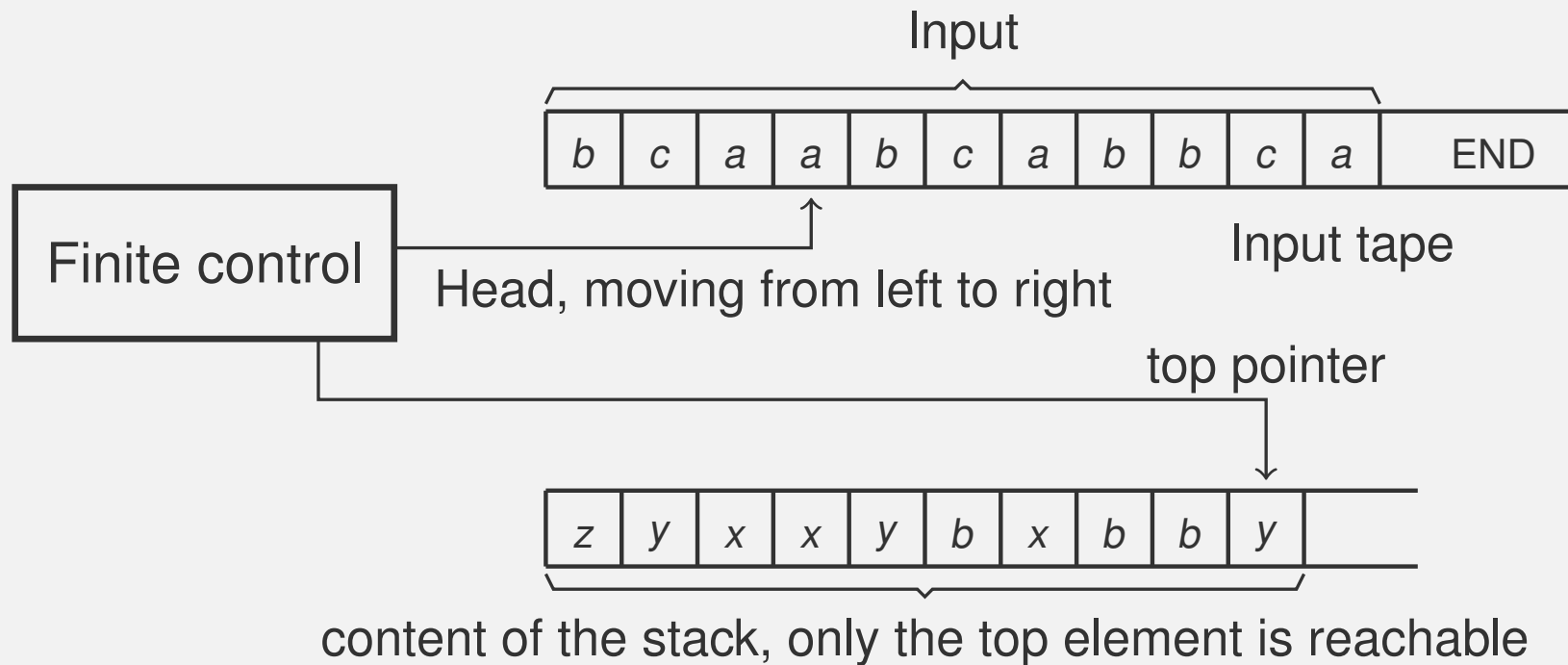


Fundamentals of theory of computation I.

Lecture 9

Krisztián Tichler
ktichler@inf.elte.hu

Pushdown automaton



- ▶ Pushdown automata are generalizations of finite automata with a **pushdown store** (or **stack**) of unlimited capacity.
- ▶ In each step some new content is pushed into the stack replacing the top element of the stack.
- ▶ They are nondeterministic by default.

Pushdown automata

Notation: Let $\mathcal{P}_{\text{fin}}(X)$ denote the set of all finite subset of a set X .

Definition

A **pushdown automaton** (PDA) is a seven-tuple

$A = (Z, Q, \Sigma, \delta, z_0, q_0, F)$, where

- ▶ Z is a finite set of pushdown symbols (*pushdown alphabet* or *stack alphabet*),
- ▶ Q is a finite set of internal states,
- ▶ Σ is a finite set of input symbols (*input alphabet*),
- ▶ $\delta : Z \times Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}_{\text{fin}}(Z^* \times Q)$ is the *transition function*,
- ▶ $z_0 \in Z$ is the initial symbol of the stack,
- ▶ $q_0 \in Q$ is the initial state,
- ▶ $F \subseteq Q$ is the set of accepting states.

Pushdown automata

- ▶ So, next configuration depends on current state, next input symbol to be processed and top element of the stack.
- ▶ In each step exactly one element is removed from the pushdown store and replaced by some $(0, 1, 2, \dots)$ elements over the pushdown alphabet.
- ▶ If $\delta(z, q, \varepsilon)$ is non-empty we have a **ε -transition** (or ε -move). In the case of an ε -move the PDA switches state and/or changes the content of the stack, but does not process any letter from the input.
- ▶ ε -moves are possible even before the first letter and even after the last letter is processed.

Configurations of pushdown automata

Definition

A **configuration** of a pushdown automaton is a word of the form zqw where

- ▶ $z \in Z^*$ is the current content of the pushdown store,
- ▶ $q \in Q$ is the current state of the finite control,
- ▶ $w \in \Sigma^*$ is the remaining suffix of the input word still to be processed.

The **initial configuration** for the input word $u \in \Sigma^*$ is z_0q_0u .

- ▶ The leftmost (first) letter of z is at the bottom of the stack, while the rightmost (last) letter of z is the top element of the pushdown store.
- ▶ The leftmost letter of w is the next letter of the input to be processed. This is the cell, where the head is.

Basic pushdown operations

Let $t \in \Sigma \cup \{\varepsilon\}$, $q, r \in Q$ and $z \in Z$

- ▶ $(\varepsilon, r) \in \delta(z, q, t)$: removes z from the stack (POP operation)
- ▶ $(z, r) \in \delta(z, q, t)$: does not change the content of the stack
- ▶ $(z', r) \in \delta(z, q, t)$: the topmost letter z is overwritten by z' in the stack ($z' \in Z$)
- ▶ $(zz', r) \in \delta(z, q, t)$: z' has been put on the top of the stack (on top of z) ($z' \in Z$) (PUSH operation)
- ▶ Other possibilities, such $(zz'z'', r) \in \delta(z, q, t)$: $z'z''$ has been put on the top of the stack, z'' is the new top element ($z', z'' \in Z$) .
- ▶ More generally $(w, r) \in \delta(z, q, t)$, where $w \in Z^*$ is an arbitrary word over Z . w is put in place of z , the last letter of w is the new top element of the pushdown store.

One step reduction

Definition

A pushdown automaton A **reduces** a configuration $x \in Z^* Q \Sigma^*$ to a configuration $y \in Z^* Q \Sigma^*$ **in one move**, denoted by $x \Rightarrow_A y$, iff there are $z \in Z, q, p \in Q, a \in \Sigma \cup \{\varepsilon\}, r, u \in Z^*$ and $w \in \Sigma^*$ such that $(u, p) \in \delta(z, q, a)$ with $x = rzqa w$ and $y = rupw$.

Examples:

- ▶ If $\delta(c, q_1, a) = \{(dd, q_2), (\varepsilon, q_4)\}$ in PDA A and $z_0cddcq_1ababba$ is a configuration then both $z_0cddcq_1ababba \Rightarrow_A z_0cddddq_2babba$ and $z_0cddcq_1ababba \Rightarrow_A z_0cddq_4babba$ holds.
- ▶ If $\delta(c, q_3, \varepsilon) = \{(dd, q_2)\}$ in PDA A and $z_0cddcq_3ababba$ is a configuration then $z_0cddcq_3ababba \Rightarrow_A z_0cddddq_2ababba$ holds.
- ▶ If $\delta(c, q_5, \varepsilon) = \emptyset$ and $\delta(c, q_5, a) = \emptyset$ in PDA A , then there is no configuration C with the property $z_0ccq_5aab \Rightarrow_A C$.

Multistep reduction, recognized language

Definition

A pushdown automaton A **reduces** a configuration $x \in Z^* Q \Sigma^*$ to a configuration $y \in Z^* Q \Sigma^*$, denoted by $x \Rightarrow_A^* y$, iff either $x = y$ or there exists a finite sequence of words $x_1, \dots, x_n \in Z^* Q \Sigma^*$ such that $x = x_1$, $y = x_n$, and $x_i \Rightarrow_A x_{i+1}$ holds for all $1 \leq i \leq n - 1$.

So $\Rightarrow_A^* \subseteq Z^* Q \Sigma^* \times Z^* Q \Sigma^*$ is the reflexive, transitive closure of $\Rightarrow_A$.

Example: If both $\delta(d, q_6, b) = \{(\varepsilon, q_5)\}$ and $\delta(d, q_5, \varepsilon) = \{(dd, q_2), (\varepsilon, q_4)\}$ holds, then

$\#cddq_6bab \Rightarrow_A \#cdq_5ab \Rightarrow_A \#cddq_2ab$ and

$\#cddq_6bab \Rightarrow_A \#cdq_5ab \Rightarrow_A \#cq_4ab$.

So $\#cddq_6bab \Rightarrow_A^* \#cddq_2ab$ and $\#cddq_6bab \Rightarrow_A^* \#cq_4ab$.

Definition

The **language accepted by a pushdown automaton**

$A = (Z, Q, \Sigma, \delta, z_0, q_0, F)$ **by accepting states** is defined as

$L(A) = \{u \in \Sigma^* \mid z_0 q_0 u \Rightarrow_A^* wp \text{ for some } w \in Z^*, p \in F\}$.

Nondeterministic behavior

Let $a \in \Sigma$, $q \in Q$ be $z \in Z$ arbitrary. Assume, that $\delta(z, q, a) = \{(u_1, r_1), \dots, (u_k, r_k)\}$ and $\delta(z, q, \varepsilon) = \{(u_{k+1}, r_{k+1}), \dots, (u_n, r_n)\}$ hold, where $k \leq n$, $u_i \in Z^*$ and $r_i \in Q$, $1 \leq i \leq n$.

Whenever the head of a pushdown automaton A is on the input symbol $a \in \Sigma$ in the state $q \in Q$ and reading $z \in Z$ from the stack, then the next state of A is one of the r_i 's ($1 \leq i \leq n$). z is replaced by u_i . In the case of $1 \leq i \leq k$ the head moves one cell to the right, while in the case of $k + 1 \leq i \leq n$ the head does not move.

For any deterministic automaton n should be equal to 1 for all $a \in \Sigma$, $q \in Q$ and $z \in Z$.

Deterministic pushdown automata

Definition

A PDA $A = (Z, Q, \Sigma, \delta, z_0, q_0, F)$ is said to be **deterministic** if $|\delta(z, q, a)| + |\delta(z, q, \varepsilon)| = 1$ holds for all $(z, q, a) \in Z \times Q \times \Sigma$.

So for every $q \in Q$ and $z \in Z$ we have

- ▶ either $\delta(z, q, a)$ contains exactly one element for every $a \in \Sigma$ and $\delta(z, q, \varepsilon) = \emptyset$,
- ▶ or $\delta(z, q, \varepsilon)$ contains exactly one element and $\delta(z, q, a) = \emptyset$ for every input symbol $a \in \Sigma$.

Observation: If $|\delta(z, q, a)| + |\delta(z, q, \varepsilon)| \leq 1$ holds for every $(z, q, a) \in Z \times Q \times \Sigma$ then the automaton can be easily extended to be deterministic by adding a new state and let missing transitions end up in the new state without modifying the content of the stack.

So pushdown automata satisfying this weaker condition can be considered to as being deterministic as well.

Alternative representations

► Derivation rules:

$zqa \rightarrow up$ is a rule $\iff (u, p) \in \delta(z, q, a)$,

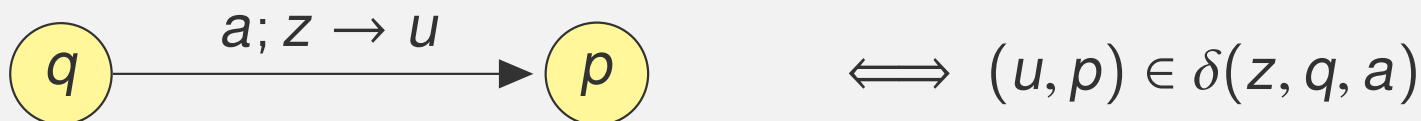
$zq \rightarrow up$ is a rule $\iff (u, p) \in \delta(z, q, \varepsilon)$.

$(p, q \in Q, a \in \Sigma, z \in Z, u \in Z^*)$

The set of derivation rules corresponding to transition function δ is denoted by M_δ .

► Transition diagram:

in the case of $p, q \in Q, a \in \Sigma \cup \{\varepsilon\}, z \in Z, u \in Z^*$:



Accepting states are double circled, the initial state is denoted by a small $\rightarrow$.

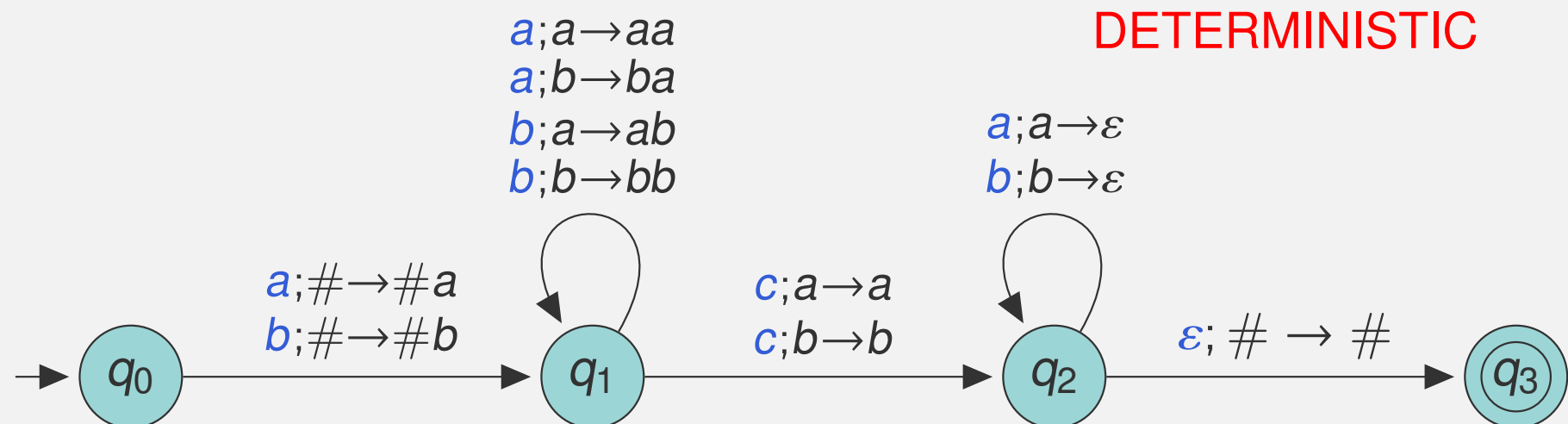
Pushdown automata

Example 1: Let $L_1 = \{wcw^R \mid w \in \{a, b\}^+\}$. Construct a PDA A with the property $L(A) = L_1$.

Solution:

$A = (\{\#, a, b\}, \{q_0, q_1, q_2, q_3\}, \{a, b, c\}, \delta, \#, q_0, \{q_3\})$, where

$$\begin{aligned}(\#t, q_1) &\in \delta(\#, q_0, t) & \forall t \in \{a, b\} \\(zt, q_1) &\in \delta(z, q_1, t) & \forall z, t \in \{a, b\} \\(z, q_2) &\in \delta(z, q_1, c) & \forall z \in \{a, b\} \\(\varepsilon, q_2) &\in \delta(t, q_2, t) & \forall t \in \{a, b\} \\(\#, q_3) &\in \delta(\#, q_2, \varepsilon)\end{aligned}$$



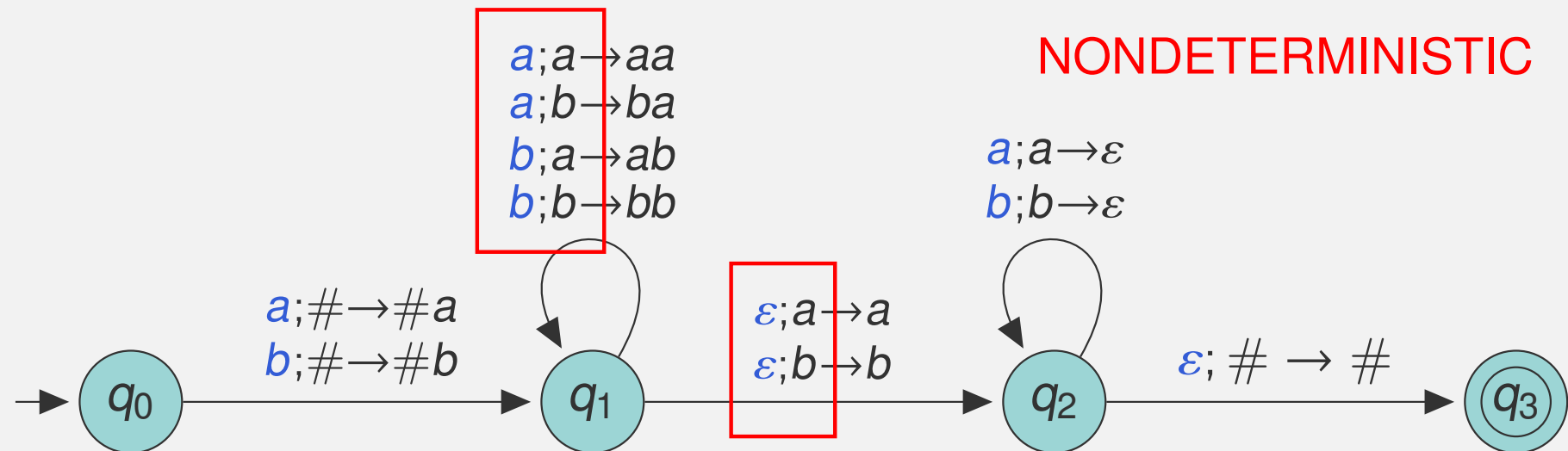
Pushdown automata

Example 2: Let $L_2 = \{ww^R \mid w \in \{a, b\}^+\}$. Construct a PDA A with the property $L(A) = L_2$.

Solution:

$A = (\{\#, a, b\}, \{q_0, q_1, q_2, q_3\}, \{a, b\}, \delta, \#, q_0, \{q_3\})$, where

$$\begin{aligned}(\#t, q_1) &\in \delta(\#, q_0, t) & \forall t \in \{a, b\} \\(zt, q_1) &\in \delta(z, q_1, t) & \forall z, t \in \{a, b\} \\(z, q_2) &\in \delta(z, q_1, \varepsilon) & \forall z \in \{a, b\} \\(\varepsilon, q_2) &\in \delta(t, q_2, t) & \forall t \in \{a, b\} \\(\#, q_3) &\in \delta(\#, q_2, \varepsilon)\end{aligned}$$



Pushdown automata

Let $\mathcal{L}_{\text{detPDA}}$ denote the set of languages recognizable by a deterministic PDA by accepting states. Let $\mathcal{L}_{\text{PDA}}$ denote the set of languages recognizable by a PDA. Since every deterministic PDA is a PDA, we have $\mathcal{L}_{\text{detPDA}} \subseteq \mathcal{L}_{\text{PDA}}$.

Theorem

$$\mathcal{L}_3 \subset \mathcal{L}_{\text{detPDA}} \subset \mathcal{L}_{\text{PDA}}$$

We do not prove this theorem. Note, that $L_1 = \{wcw^R \mid w \in \{a, b\}^+\}$ is non-regular (can be checked by Myhill-Nerode theorem, the languages L_{a^n} ($n \geq 0$) are pairwise disjoint). We've just seen, that L_1 can be recognized by a deterministic PDA.

On the other hand $L_2 = \{ww^R \mid w \in \{a, b\}^+\}$ can be recognized by a PDA, but can not be recognized by a deterministic PDA. (This is the missing part from the proof.)

Accepting by empty store

Definition

The **language** $N(A)$ **accepted by a pushdown automaton** A **by empty store** is defined as

$$N(A) = \{u \in \Sigma^* \mid z_0 q_0 u \xRightarrow{*}_A p \text{ for some } p \in Q\}.$$

Remark:

Observe, that runs are blocked whenever a the stack becomes empty. (see the definition of δ).

So, to accept a word by empty store, the stack should become empty only in the last move.

PDA's were defined to start with a symbol in the store to avoid blocking right in the first step.

Remark: Observe, that the set F is irrelevant in the case of accepting by empty store.

Accepting by empty store

Exercise 3: Make a PDA accepting $L = \{a^n b^n \mid n \geq 1\}$ by empty store.

Solution: For PDA $A = (\{\$, a\}\{q_0, q_1\}, \{a, b\}, \delta, \$, q_0, \{\})$ we have $N(A) = \{a^n b^n \mid n \geq 1\}$.

The set of derivation rules:

$$\$q_0 a \rightarrow \$aq_0$$

$$aq_0 a \rightarrow aaq_0$$

$$aq_0 b \rightarrow q_1$$

$$aq_1 b \rightarrow q_1$$

$$\$q_1 \rightarrow q_1.$$

A is deterministic. For $a^2 b^3$ we have

$$\$q_0 aabbb \Rightarrow \$aq_0 abbb \Rightarrow \$aaq_0 bbb \Rightarrow \$aq_1 bb \Rightarrow \$q_1 b \Rightarrow q_1 b,$$

so A rejects $aabbb$. Although the stack is empty but the input is not fully processed.

Accept states vs empty stores

Lemma

For every pushdown automaton A there is a pushdown automaton A' with $N(A') = L(A)$.

Proof (sketch)

Let $A = (Z, Q, \Sigma, \delta, z_0, q_0, F)$ be pushdown automaton. Let $z'_0 \notin Z, q'_0, q'_h \notin Q$. Let transition function of $A' = (Z \cup \{z'_0\}, Q \cup \{q'_0, q'_h\}, \Sigma, \delta', z'_0, q'_0, \{\})$ be defined as follows.

$$\begin{aligned}\delta'(z'_0, q'_0, \varepsilon) &:= \{(z'_0 z_0, q_0)\}, \\ \delta'(z, q, a) &:= \delta(z, q, a), \quad z \in Z, q \in Q, a \in \Sigma, \\ \delta'(z, q, \varepsilon) &:= \supseteq \delta(z, q, \varepsilon), \quad z \in Z, q \in Q, \\ (\varepsilon, q'_h) &\in \delta'(z, q, \varepsilon), \quad z \in Z \cup \{z'_0\}, q \in F \cup \{q'_h\}.\end{aligned}$$

A' can simulate A . $z'_0 \neq z_0$ is needed to avoid acceptance if A makes its store empty without the input being fully processed.

Accept states vs empty stores

Let $w \in L(A)$, so $z_0 q_0 w \Rightarrow_A^* uq$ holds for some $u \in Z^*$, $q \in F$. Then

$$z'_0 q'_0 w \Rightarrow_{A'}, z'_0 z_0 q_0 w \Rightarrow_{A'}^*, z'_0 uq \Rightarrow_{A'}^*, z'_0 q'_h \Rightarrow_{A'} q'_h,$$

holds, so $w \in N(A')$.

Let $w \in N(A')$, so $z'_0 q'_0 w \Rightarrow_{A'}^* p$ holds for some $p \in Q \cup \{q'_0, q'_h\}$.

The first step should be $z'_0 q'_0 w \Rightarrow_{A'} z'_0 z_0 q_0 w$. Since z'_0 can be removed from the store only by a transition $(\varepsilon, q'_h) \in \delta'(z'_0, q, \varepsilon)$ we have $p = q'_h$ and there should be $q \in F, z \in Z, u \in Z^*$ with

$$z'_0 z_0 q_0 w \Rightarrow_{A'}^*, z'_0 uzq \Rightarrow_{A'}, z'_0 uq'_h \Rightarrow_{A'}^*, z'_0 q'_h \Rightarrow_{A'} q'_h.$$

The first $\Rightarrow_{A'}^*$ uses only transitions from A , so we have $z_0 q_0 w \Rightarrow_A^* uzq$ implying $w \in L(A)$.

Accept states vs empty stores

Lemma

For every pushdown automaton A there is a pushdown automaton A' with $L(A') = N(A)$.

Proof (sketch) Let $A = (Z, Q, \Sigma, \delta, z_0, q_0, \{\})$ be a pushdown automaton. We construct a PDA A' with the property $L(A') = N(A)$. Let $z'_0 \notin Z$, $q'_0, q'_f \notin Q$ and $A' = (Z \cup \{z'_0\}, Q \cup \{q'_0, q'_f\}, \Sigma, \delta', z'_0, q'_0, \{q'_f\})$.

$$\begin{aligned}\delta'(z'_0, q'_0, \varepsilon) &:= \{(z'_0 z_0, q_0)\}, \\ \delta'(z, q, a) &:= \delta(z, q, a), \quad z \in Z, q \in Q, a \in (\Sigma \cup \{\varepsilon\}), \\ \delta'(z'_0, q, \varepsilon) &:= \{(z'_0, q'_f)\}, \quad q \in Q.\end{aligned}$$

It is easy to see, that whenever the store of A becomes empty A' goes to its accepting state and A' goes to its accepting state only in these situations. Since there are no further transitions from q'_f we have $L(A') = N(A)$.

PDA vs CF grammars

Theorem

For every context free grammar G there is a pushdown automaton A with $L(A) = L(G)$.

Proof (sketch)

Let $G = (N, \Sigma, P, S)$ be a grammar in Chomsky normal form (every CF grammar can be transformed to Chomsky NF). If $\varepsilon \in L(G)$, then P includes $S \rightarrow \varepsilon$, but S does not appear on any right hand sides.

We construct a PDA A with the property $L(A) = L(G)$ teljesül.

PDA vs CF grammars

Let $A = (N \cup \{z_0\}, Q, \Sigma, \delta, z_0, q_0, \{q_h\})$, where

$$Q = \bigcup_{X \in N} \{q_X\} \cup \{q_0, q_h\}$$

and δ is defined as follows.

- (1) $z_0 q_0 \rightarrow z_0 q_S : \in M_\delta$ iff $S \rightarrow \varepsilon \in P$,
- (2) $z_0 q_0 a \rightarrow z_0 q_X : \in M_\delta$ iff $X \rightarrow a \in P$,
- (3) $Z q_Y a \rightarrow Z Y q_X : \in M_\delta$ for all $Z \in N \cup \{z_0\}$,
 $Y \in N, X \rightarrow a \in P$,
- (4) $Z q_Y \rightarrow q_X : \in M_\delta$ iff $X \rightarrow ZY \in P$,
- (5) $z_0 q_S \rightarrow q_h : \in M_\delta$.

PDA vs CF grammars

The rules M_δ are essentially the inverted rules of G .

E.g., if $G = (\{S, X, Y, Z, A, B\}, \{a, b\}, \{S \rightarrow XY, X \rightarrow AB, Y \rightarrow ZB, Z \rightarrow BA, A \rightarrow a, B \rightarrow b\}, S)$, then the following **rightmost derivation** of *abbab* (we always choose to rewrite the rightmost nonterminal of the sentential form)

$$S \Rightarrow XY \Rightarrow \textcolor{green}{XZB} \Rightarrow \textcolor{red}{XZb} \Rightarrow \textcolor{blue}{XBAb} \Rightarrow$$

$$\textcolor{blue}{XBab} \Rightarrow \textcolor{blue}{Xbab} \Rightarrow \textcolor{blue}{ABbab} \Rightarrow \textcolor{blue}{Abbab} \Rightarrow \textcolor{blue}{abbab}$$

corresponds to

$$z_0 q_0 \textcolor{blue}{abbab} \Rightarrow z_0 q_A \textcolor{blue}{bbab} \Rightarrow z_0 A q_B \textcolor{blue}{bab} \Rightarrow z_0 q_X \textcolor{blue}{bab} \Rightarrow z_0 X q_B \textcolor{blue}{ab} \Rightarrow$$

$$z_0 \textcolor{blue}{XB} q_A \textcolor{blue}{b} \Rightarrow z_0 \textcolor{red}{X} q_Z \textcolor{red}{b} \Rightarrow z_0 \textcolor{green}{XZ} q_B \Rightarrow z_0 X q_Y \Rightarrow z_0 q_S$$

and vice versa.

In general it can be proved by induction that

$$z_0 q_0 u \Rightarrow_A^* z_0 W q_X v \iff WXv \Rightarrow_G^* u \text{ holds, where}$$

$W \in N^*$, $X \in N$, $v \in \Sigma^*$ and v is a suffix of u . (it holds for $u \neq \varepsilon$ due to (2),(3),(4), and for $u = \varepsilon$ as well due to (1).)

PDA vs CF grammars

We have the following for $v = \varepsilon$, $W = \varepsilon$, $X = S$.

$$z_0 q_0 u \Rightarrow_A^* z_0 q_S \iff S \Rightarrow_G^* u.$$

Let $u \in L(A)$. Then $z_0 q_0 u \Rightarrow_A^* z q_h$ holds for some $z \in Z^*$.

By the definition of M_δ $z = \varepsilon$ should hold, more precisely

$$z_0 q_0 u \Rightarrow_A^* z_0 q_S \Rightarrow_A q_h.$$

So $S \Rightarrow_G^* u$ holds implying $L(A) \subseteq L(G)$.

If $S \Rightarrow_G^* u$ holds, then $z_0 q_0 u \Rightarrow_A^* z_0 q_S$. Due to (5) we have $z_0 q_0 u \Rightarrow_A^* q_h$ implying $L(G) \subseteq L(A)$.

PDA vs CF grammars

Theorem

For every pushdown automaton A there exists a context-free grammar G with the property $L(G) = N(A)$.

Proof (sketch)

Let $A = (Z, Q, \Sigma, \delta, z_0, q_0, \{ \})$ be a pushdown automaton.

Let the nonterminals of the grammar $G = (N, \Sigma, P, S)$ be tuples $[q, x, p]$ where $q, p \in Q$ and $x \in Z$ and let N contain a symbol S , i.e., let $N = Q \times Z \times Q \cup \{S\}$.

PDA vs CF grammars

We define the set of rules P as follows.

- ▶ Let $S \rightarrow [q_0, z_0, p] \in P$ for every $p \in Q$.
- ▶ If $xqa \rightarrow y_1 \cdots y_m p_m \in M_\delta$ holds for some $a \in (\Sigma \cup \{\varepsilon\})$, then for any sequence $p_0, p_1, \dots, p_{m-1} \in Q$ of states let

$$[q, x, p_0] \rightarrow a[p_m, y_m, p_{m-1}] \cdots [p_1, y_1, p_0]$$

be a rule of P .

E.g., if $m = 0$ i.e., $xqa \rightarrow p_0 \in M_\delta$, then let $[q, x, p_0] \rightarrow a \in P$.

- ▶ There are no further rules in P .

PDA vs CF grammars

To prove the inclusion $L(G) \subseteq N(A)$ we prove the following claim.
For every symbol x of the stack alphabet, states q, p and input word u we have $[q, x, p] \Rightarrow_G^* u$ if and only if $xqu \Rightarrow_A^* p$.

We prove the claim by induction on the length of the derivation $[q, x, p] \Rightarrow_G^* u$.

In the case of a single step we have $u = a \in \Sigma \cup \{\varepsilon\}$ and $xqa \rightarrow p \in M_\delta$.

Assume that the statement holds for derivations of length at most n and let the derivation $[q, x, p_0] \Rightarrow_G^* u$ consist of $n + 1$ steps.

The derivation is of the form

$$[q, x, p_0] \Rightarrow_G a[p_m, y_m, p_{m-1}] \cdots [p_1, y_1, p_0] \Rightarrow_G^* u,$$

implying the existence of words $u_m, u_{m-1}, \dots, u_1 \in \Sigma^*$ with $u = au_mu_{m-1} \cdots u_1$ and $[p_i, y_i, p_{i-1}] \Rightarrow_G^* u_i$, $1 \leq i \leq m$.

PDA vs CF grammars

By induction we have a reduction

$$y_i p_i u_i \Rightarrow_A^* p_{i-1}$$

and by the definition of G we have

$$xqa \Rightarrow_A y_1 \cdots y_m p_m.$$

So

$$\begin{aligned} xqu = xqau_m \cdots u_1 &\Rightarrow_A y_1 \cdots y_m p_m u_m \cdots u_1 \Rightarrow_A^* \\ &y_1 \cdots y_{m-1} p_{m-1} u_{m-1} \cdots u_1 \Rightarrow_A^* p_0. \end{aligned}$$

If $u \in L(G)$, then there is a $p \in Q$ with

$$S \Rightarrow_G [q_0, z_0, p] \Rightarrow_G^* u,$$

and by the claim $z_0 q_0 u \Rightarrow_A^* p$, we have $u \in N(A)$.

PDA vs CF grammars

For the other direction we start again by proving a claim: $xqu \Rightarrow_A^* p$ implies $[q, x, p] \Rightarrow_G^* u$.

We prove the claim by induction on the number of transition steps of the reduction in A .

In the case of a single step we have $xqa \rightarrow p \in M_\delta$, implying $[q, x, p] \rightarrow a \in P$.

In the case of multiple steps the reduction $xqu \Rightarrow_A^* p$ should be of the form

$$xqu = xqav \Rightarrow_A y_1 \cdots y_m p_m v \Rightarrow_A^* p.$$

According to the definition of reductions of PDA's there should exist a configuration $y_1 \cdots y_{m-1} p_{m-1} v_1$ with $v = u_m v_1$ for some $u_m, v_1 \in \Sigma^*$ and $p_{m-1} \in Q$, i.e.,

$$xqu = xqav \Rightarrow_A y_1 \cdots y_m p_m u_m v_1 \Rightarrow_A^* y_1 \cdots y_{m-1} p_{m-1} v_1 \Rightarrow_A^* p.$$

PDA vs CF grammars

Continuing this way we get words, $u_{m-1}, \dots, u_1$ with the properties $u = au_mu_{m-1} \cdots u_1$ and

$$y_i p_i u_i \Rightarrow_A^* p_{i-1}, \text{ ha } i = 1, \dots, m$$

and $p_0 = p$. By induction we have

$$[p_i, y_i, p_{i-1}] \Rightarrow_G^* u_i, \text{ ha } i = 1, \dots, m$$

and by the definition of G we also have

$$[q, x, p] = [q, x, p_0] \Rightarrow_G a[p_m, y_m, p_{m-1}] \cdots [p_1, y_1, p_0]$$

implying $[q, x, p] \Rightarrow_G^* u$.

So, if $u \in N(A)$ holds, then we have $z_0 q_0 u \Rightarrow_A^* p$ for some state p and by the claim just proved $S \Rightarrow_G [q_0, z_0, p] \Rightarrow_G^* u$ implying $N(A) \subseteq L(G)$.

Descriptive power of PDA's – summary

We've proved the following.

For every language L the following statements are equivalent

- ▶ L is context-free, i.e., it can be generated by a type 2 grammar.
- ▶ L can be recognized by a (nondeterministic) PDA by accept states.
- ▶ L can be recognized by a (nondeterministic) PDA by empty stores.

So, if $\mathcal{L}_{\text{ePDA}}$ denotes the set of languages that can be recognized by a PDA by empty store, then we have

$$\mathcal{L}_3 \subset \mathcal{L}_{\text{detPDA}} \subset \mathcal{L}_{\text{PDA}} = \mathcal{L}_{\text{ePDA}} = \mathcal{L}_2$$

According to the connection between PDA's and CF languages the elements of $\mathcal{L}_{\text{detPDA}}$ are often called **deterministic context free** languages.